First-Step-Contraction MPC With a Bounded Frozen-`dhat` Steady-State Target

April 6, 2026

1 Purpose

This report documents the bounded frozen-`dhat` target method that is now available beside the existing refined Step A selector in the first-step-contraction MPC workflow.

The intent is narrow:

- keep the maintained upstream offset-free MPC and first-step-contraction logic unchanged,
- replace only the steady-state target package used as the Lyapunov center,
- preserve the old refined-selector path in the codebase and in the original notebook.

The new notebook uses the same upstream controller, the same replacement logic, and the same debug-export path. Only the way the steady-state package $(x_s, u_s, d_s, y_s, r_s)$ is computed changes.

2 Scaled Deviation Coordinates

All quantities below are in the same scaled deviation coordinates already used by the identified linear model, the observer, and the MPC controller. Let

$$u_k^{\text{dev}} = \text{scale}(u_k^{\text{phys}}) - u_{ss}^{\text{scaled}}, \tag{1}$$

$$y_k^{\text{dev}} = \text{scale}(y_k^{\text{phys}}) - y_{ss}^{\text{scaled}}. \tag{2}$$

The steady-state target calculation is performed entirely in these deviation coordinates. Conversion back to physical units is done only for plotting or reporting.

3 Augmented Offset-Free Model

The first-step-contraction notebook uses the same Rawlings-style output-disturbance augmentation as the maintained offset-free MPC workflow:

$$z_k = \begin{bmatrix} x_k \\ d_k \end{bmatrix}, \tag{3}$$

$$z_{k+1} = \underbrace{\begin{bmatrix} A & B_d \\ 0 & I \end{bmatrix}}_{A_{\text{aug}}} z_k + \underbrace{\begin{bmatrix} B \\ 0 \end{bmatrix}}_{B_{\text{aug}}} u_k, \tag{4}$$

$$y_k = \underbrace{\begin{bmatrix} C & C_d \end{bmatrix}}_{C_{\text{aug}}} z_k. \tag{5}$$

In the notebook configuration used for the new method:

$$C_d = I, \tag{6}$$

and the augmented state is ordered as

$$z_k = \begin{bmatrix} x_k \\ d_k \end{bmatrix}, \tag{7}$$

with the last n_y augmented states representing output disturbances.

4 Observer Split and Frozen Disturbance Assumption

At each control step the observer provides the augmented estimate

$$\hat{z}_k = \begin{bmatrix} \hat{x}_k \\ \hat{d}_k \end{bmatrix}. \tag{8}$$

The bounded frozen-`dhat` target method uses the simplest steady-state disturbance policy:

$$d_s(k) = \hat{d}_k. \tag{9}$$

So the disturbance target is *not* optimized. It is frozen at the current observer estimate.

This differs from the refined Step A selector, where the target package is produced by a separate convex selector problem with its own smoothing and anchoring penalties. Here there is no secondary selector cost shaping the steady package. The disturbance estimate itself defines the output offset.

5 Frozen-`dhat` Steady-State Equations

Because the target computation is based on the unaugmented plant model with output disturbance entering through the measurement equation, the steady-state conditions are

$$x_s = Ax_s + Bu_s, \tag{10}$$

$$y_s = Cx_s + d_s, \tag{11}$$

with

$$d_s = \hat{d}_k. \tag{12}$$

If the requested setpoint at step k is $y_{sp,k}$, then the exact offset-free target must satisfy

$$y_s = y_{sp,k}. \tag{13}$$

Substituting $d_s = \hat{d}_k$ gives

$$Cx_s = y_{sp,k} - \hat{d}_k. \tag{14}$$

Define the right-hand side

$$r_k := y_{sp,k} - \hat{d}_k. \tag{15}$$

Then the frozen-`dhat` steady-state equations become

$$(I - A)x_s - Bu_s = 0, \tag{16}$$

$$Cx_s = r_k. \tag{17}$$

6 Stacked Exact Solve

Introduce the stacked unknown

$$w_s := \begin{bmatrix} x_s \\ u_s \end{bmatrix}. \tag{18}$$

Then the steady-state equations can be written as

$$M w_s = b_k, \tag{19}$$

with

$$M = \begin{bmatrix} I - A & -B \\ C & 0 \end{bmatrix}, \tag{20}$$

$$b_k = \begin{bmatrix} 0 \\ r_k \end{bmatrix}. \tag{21}$$

The exact solution is obtained from this linear system whenever the system is square, full-rank, and sufficiently well-conditioned. In code this is the `stacked_exact` branch.

7 Reduced Exact Solve

If $(I - A)$ is safely invertible, then

$$x_s = (I - A)^{-1} B u_s. \tag{22}$$

Substituting this into the output equation yields the reduced map

$$G := C(I - A)^{-1} B, \tag{23}$$

and the reduced steady-state equation

$$G u_s = r_k. \tag{24}$$

If G is square, full-rank, and well-conditioned, the exact target is

$$u_s = G^{-1} r_k, \qquad x_s = (I - A)^{-1} B u_s. \tag{25}$$

The code uses the following exact-solve priority:

1. `stacked_exact` if the stacked system is square, full-rank, and well-conditioned,
2. `reduced_exact` if the reduced map is available and well-conditioned,
3. least-squares fallbacks only if the exact modes are not numerically trustworthy.

8 Why a Box-Constrained Fallback Is Needed

The exact frozen-`dhat` target may violate the input box

$$u_{\min} \le u_s \le u_{\max}. \tag{26}$$

When this happens, the exact target is not an admissible steady-state center for a constrained controller. It is still diagnostically useful, but it is not a proper feasible target.

That is why the second stage solves a simple bounded optimization problem.

9 Bounded Least-Squares Fallback

The simplest bounded version of the target problem is

$$\min_{x_s,u_s} \quad \left\| \begin{bmatrix} I-A & -B \\ C & 0 \end{bmatrix} \begin{bmatrix} x_s \\ u_s \end{bmatrix} - \begin{bmatrix} 0 \\ r_k \end{bmatrix} \right\|_2^2 \qquad \text{subject to } u_{\min} \le u_s \le u_{\max}. \tag{27}$$

If the reduced map is available, the same problem can be written more compactly as

$$\min_{u_s} \quad \|Gu_s - r_k\|_2^2 \qquad \text{subject to } u_{\min} \le u_s \le u_{\max}, \tag{28}$$

and then

$$x_s = (I-A)^{-1}Bu_s. \tag{29}$$

This is exactly the "simplest optimization version" described in the design discussion.

10 Projection Interpretation

Assume the reduced map G is invertible and define the unbounded exact target

$$u_s^{\text{exact}} = G^{-1}r_k. \tag{30}$$

Then the bounded objective becomes

$$\|Gu - r_k\|_2^2 = \|G(u - u_s^{\text{exact}})\|_2^2 = (u - u_s^{\text{exact}})^\top G^\top G(u - u_s^{\text{exact}}). \tag{31}$$

So the bounded fallback is the projection of the exact target onto the admissible input box under the metric

$$H = G^\top G. \tag{32}$$

This explains an important practical point:

- if the exact target lies inside the box, the bounded and exact solutions coincide,
- if the exact target lies outside the box, the bounded solution often lands on a box face or corner.

11 KKT Conditions and Active Bounds

For the reduced bounded problem define

$$J(u) = \frac{1}{2}\|Gu - r_k\|_2^2. \tag{33}$$

Its gradient is

$$\nabla J(u) = G^\top(Gu - r_k). \tag{34}$$

The Karush–Kuhn–Tucker conditions at an optimum $u^\star$ are

$$u_{\min} \le u^\star \le u_{\max}, \tag{35}$$
$$G^\top(Gu^\star - r_k) + \lambda^+ - \lambda^- = 0, \tag{36}$$
$$\lambda^+ \ge 0, \qquad \lambda^- \ge 0, \tag{37}$$
$$\lambda_i^+(u_i^\star - u_{\max,i}) = 0, \tag{38}$$
$$\lambda_i^-(u_{\min,i} - u_i^\star) = 0. \tag{39}$$

These conditions explain why a bound-active steady-state target is expected in disturbance cases: if the exact target is outside the box, the optimal constrained solution is often characterized by nonzero multipliers on active bounds.

12 Returned Steady-State Package

The new target helper returns a package compatible with the first-step-contraction workflow:

$$x_s, \qquad u_s, \qquad d_s = \hat{d}_k, \qquad y_s = Cx_s + d_s, \qquad r_s = y_s. \tag{40}$$

It also returns

$$x_s^{\text{aug}} = \begin{bmatrix} x_s \\ d_s \end{bmatrix}, \tag{41}$$

plus diagnostics such as

$$\|r_s - y_{sp,k}\|_\infty, \quad \|(I-A)x_s - Bu_s\|_\infty, \quad \text{bound violation}, \quad \text{exact-within-bounds flag}, \quad \text{bounded residual norm}. \tag{42}$$

The key semantic choices are:

- `d_s_frozen = True`,
- `d_s_optimized = False`,
- `d_s_minus_dhat_inf = 0`.

13 How the New Target Enters the First-Step Controller

The first-step-contraction controller still follows the maintained architecture:

1. compute a target package,
2. build the effective target using current target or last valid target,
3. solve the ordinary upstream offset-free MPC candidate,
4. check first-step Lyapunov contraction relative to the effective target,
5. if needed, solve a constrained replacement MPC problem with one additional first-step Lyapunov inequality.

Only Step 1 changes.

14 Upstream Offset-Free MPC Candidate

With the augmented model

$$z_{j+1|k} = A_{\text{aug}} z_{j|k} + B_{\text{aug}} u_{c(j)|k}, \qquad c(j) = \min(j, N_C - 1), \tag{43}$$

the predicted output is

$$y_{j|k} = C_{\text{aug}} z_{j|k}. \tag{44}$$

The upstream controller still solves the baseline offset-free MPC objective

$$J_{\text{MPC}}(U) = \sum_{j=1}^{N_P} \|y_{j|k} - y_{\text{track},k}\|_{Q_y}^2 + \sum_{j=0}^{N_C-1} \|\Delta u_{j|k}\|_{R_{\Delta u}}^2. \tag{45}$$

In the new notebook,

$$y_{\text{track},k} = y_{sp,k}, \tag{46}$$

because both `mpc_target_policy` and `tracking_target_policy` remain `raw_setpoint`.

So the new steady-state method affects the Lyapunov center, not the nominal tracking objective of the upstream MPC.

15 Lyapunov Function

Let $P_x \succ 0$ be the physical-state Lyapunov matrix used by the maintained first-step-contraction code. Define the physical-state error relative to the effective target:

$$e_{x,k} = \hat{x}_k - x_s^{\text{eff}}. \tag{47}$$

The Lyapunov function is

$$V_k = e_{x,k}^\top P_x e_{x,k}. \tag{48}$$

The contraction bound is

$$V_{\text{bound},k} = \rho_{\text{lyap}} V_k + \varepsilon_{\text{lyap}}, \qquad 0 < \rho_{\text{lyap}} < 1. \tag{49}$$

16 First-Step Contraction Inequality

For a candidate first move $u_{0|k}$, the predicted first physical-state step is

$$\hat{x}_{k+1|k}^{(1)} = A\hat{x}_k + Bu_{0|k} + B_d \hat{d}_k, \tag{50}$$

using the augmented model that the controller itself predicts with.

The corresponding predicted Lyapunov value is

$$V_{k+1|k}^{(1)} = (\hat{x}_{k+1|k}^{(1)} - x_s^{\text{eff}})^\top P_x (\hat{x}_{k+1|k}^{(1)} - x_s^{\text{eff}}). \tag{51}$$

The maintained first-step condition is

$$V_{k+1|k}^{(1)} \le \rho_{\text{lyap}} V_k + \varepsilon_{\text{lyap}}. \tag{52}$$

If the upstream candidate already satisfies this inequality, it is accepted unchanged. If not, the controller triggers the replacement solve.

17 Replacement Solve

The replacement controller keeps the same MPC objective and the same move/input constraints as the upstream problem, but adds the single extra hard first-step constraint

$$V^{(1)}_{k+1|k} \leq \rho_{\text{lyap}} V_k + \varepsilon_{\text{lyap}}. \tag{53}$$

So the modified optimization is still

$$\min_{U} \quad J_{\text{MPC}}(U) \tag{54}$$

subject to

$$z_{j+1|k} = A_{\text{aug}} z_{j|k} + B_{\text{aug}} u_{c(j)|k}, \tag{55}$$

$$u_{\min} \leq u_{j|k} \leq u_{\max}, \tag{56}$$

$$V^{(1)}_{k+1|k} \leq \rho_{\text{lyap}} V_k + \varepsilon_{\text{lyap}}. \tag{57}$$

There is still

- no QCQP projection stage,
- no post-hoc safety filter,
- no change to the upstream stage cost,
- no change to the candidate reuse and backup logic.

18 Nominal Versus Disturbance Interpretation

The nominal case and the disturbance case behave differently even if the target solver is implemented correctly.

18.1 Nominal Case

If

$$\hat{d}_k \approx 0, \tag{58}$$

then

$$r_k = y_{sp,k} - \hat{d}_k \approx y_{sp,k}. \tag{59}$$

If the loop settles, both the plant and the frozen-`dhat` target tend toward the same nominal equilibrium. In that case it is common to see

$$u_{\text{applied},k} \to u_s(k). \tag{60}$$

18.2 Disturbance Case

In the disturbance case, the target is driven by

$$r_k = y_{sp,k} - \hat{d}_k. \tag{61}$$

Even when r_k becomes small, the reduced map may amplify that small residual:

$$u_s = G^{-1} r_k. \tag{62}$$

If G^{-1} has large entries, then small changes in r_k can still produce visible motion in u_s.

Also, the controller does *not* track u_s directly. The upstream MPC still tracks the raw output setpoint, while the bounded frozen-`dhat` target is used only as the Lyapunov center. Therefore there is no reason to expect

$$u_{\text{applied},k} = u_s(k) \tag{63}$$

in the disturbance case, even near apparent steady output behavior.

19 Algorithmic Summary

At each control step k the new target-generation branch executes:

1. Split $\hat{z}_k$ into $\hat{x}_k$ and $\hat{d}_k$.
2. Form $r_k = y_{sp,k} - \hat{d}_k$.
3. Solve the exact frozen-`dhat` steady-state equations using the unaugmented A, B, C.
4. Check whether the exact input target lies inside the input box.
5. If yes, return the exact target package.
6. If no, solve the bounded least-squares fallback and return the bounded target package.
7. Hand that package to the existing effective-target, upstream MPC, and first-step-contraction logic.

20 Code-Level Mapping

The implementation is split as follows:

- `Lyapunov/frozen_dhat_target.py`: computes the bounded frozen-`dhat` target package.
- `Simulation/run_mpc_first_step_contraction.py`: adds a new `target_generation_mode` branch.
- `LyapunovFirstStepContractionMPC_BoundedFrozenDhat.ipynb`: uses the new branch.
- `LyapunovFirstStepContractionMPC.ipynb`: remains on the refined-selector path.

21 Conclusion

The bounded frozen-`dhat` method is intentionally simpler than the refined Step A selector. It produces the Lyapunov center by solving the frozen-disturbance steady-state equations directly, then projecting onto the admissible input box when the exact target is infeasible. This gives a target that is easier to interpret mathematically:

- exact equilibrium if the box permits it,
- closest feasible equilibrium surrogate in the least-squares sense otherwise.

The rest of the first-step-contraction MPC architecture remains unchanged.